

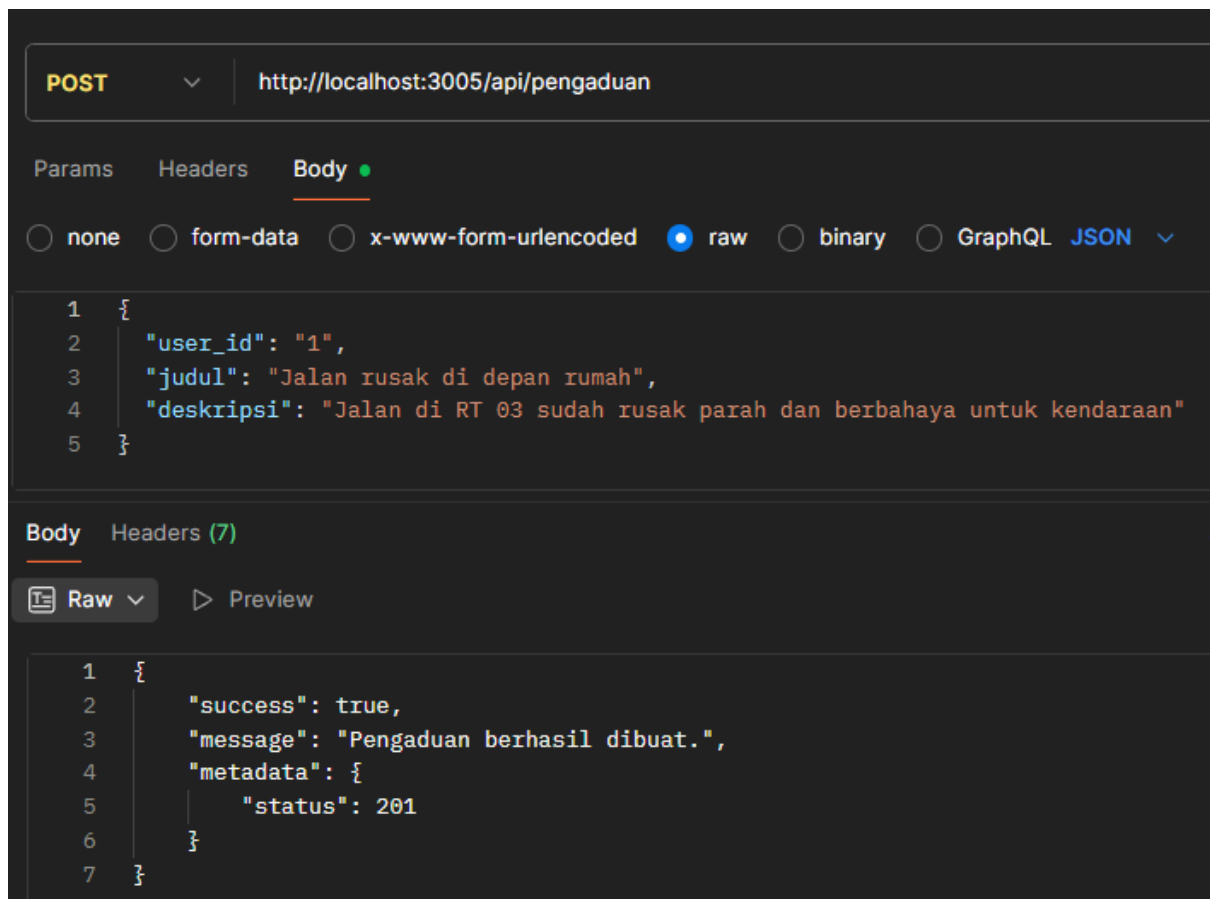
Nama: Surya Agung Firdaus

NPM; 23312006

Kelas: PBS IF 23 FX

API/pengaduan

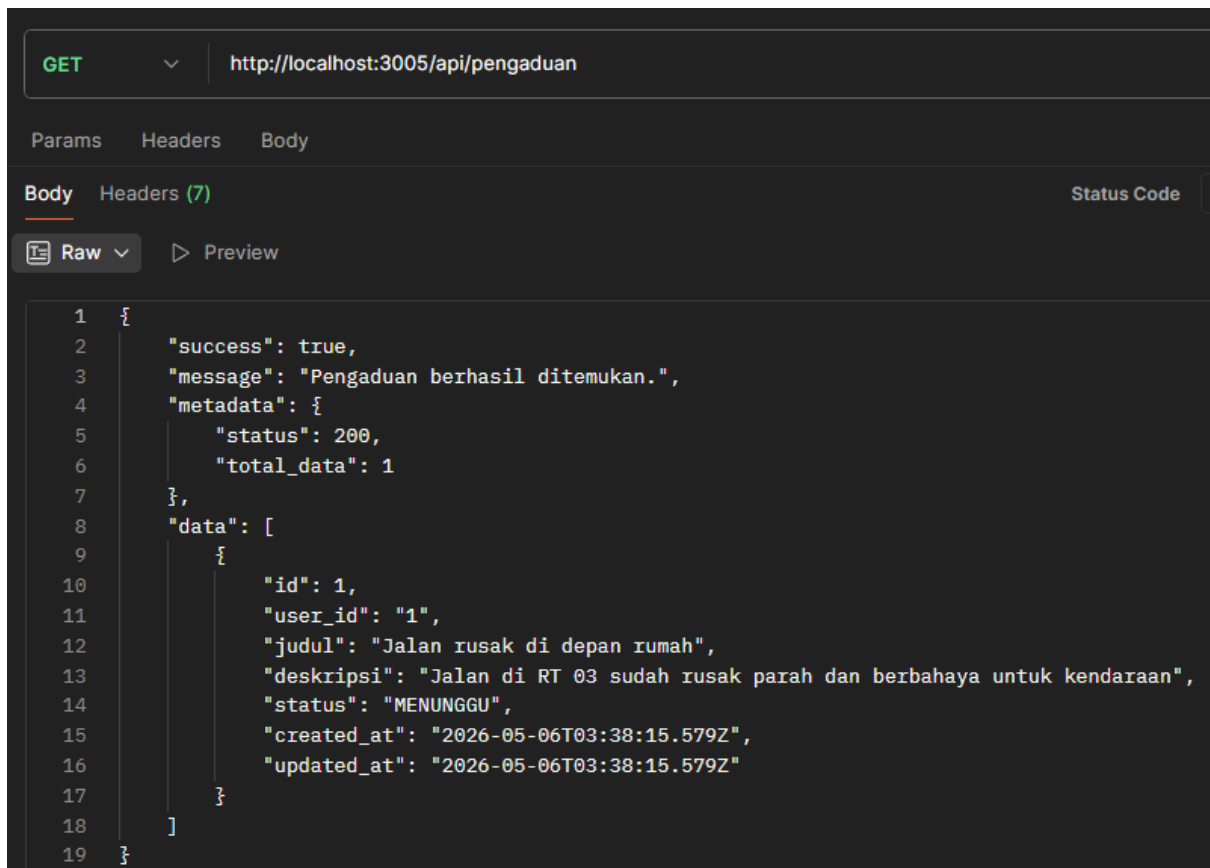
- POST



Ini adalah respon dari API POST pengaduan dengan port 3005. Pengaduan memiliki relasi dengan user. Jadi untuk bisa mengakses pengaduan dibutuhkan service user yang sedang berjalan. Untuk membuat sebuah pengaduan, diperlukan

- user_id
- judul
- deskripsi

- GET ALL

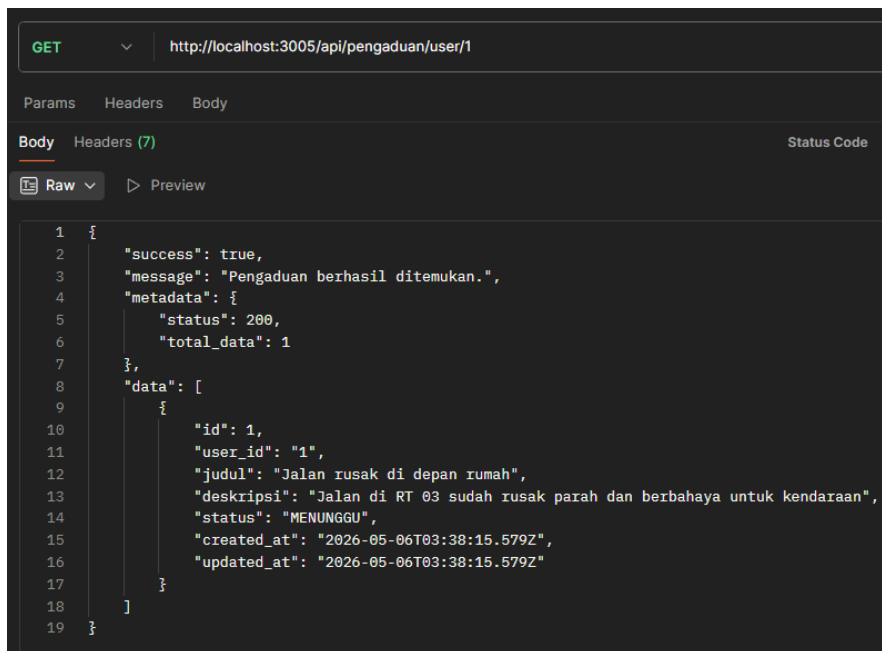


The screenshot shows a REST client interface with a GET request to `http://localhost:3005/api/pengaduan`. The response body is displayed in a raw format, showing a JSON object with a success message, metadata, and a list of complaint data.

```
1  {
2    "success": true,
3    "message": "Pengaduan berhasil ditemukan.",
4    "metadata": {
5      "status": 200,
6      "total_data": 1
7    },
8    "data": [
9      {
10       "id": 1,
11       "user_id": "1",
12       "judul": "Jalan rusak di depan rumah",
13       "deskripsi": "Jalan di RT 03 sudah rusak parah dan berbahaya untuk kendaraan",
14       "status": "MENUNGGU",
15       "created_at": "2026-05-06T03:38:15.579Z",
16       "updated_at": "2026-05-06T03:38:15.579Z"
17     }
18   ]
19 }
```

Ini adalah respon dari API GET pengaduan. Disini tidak seperti post yang membutuhkan body atau data untuk dikirimkan, kita hanya perlu memasukkan url dari request ini.

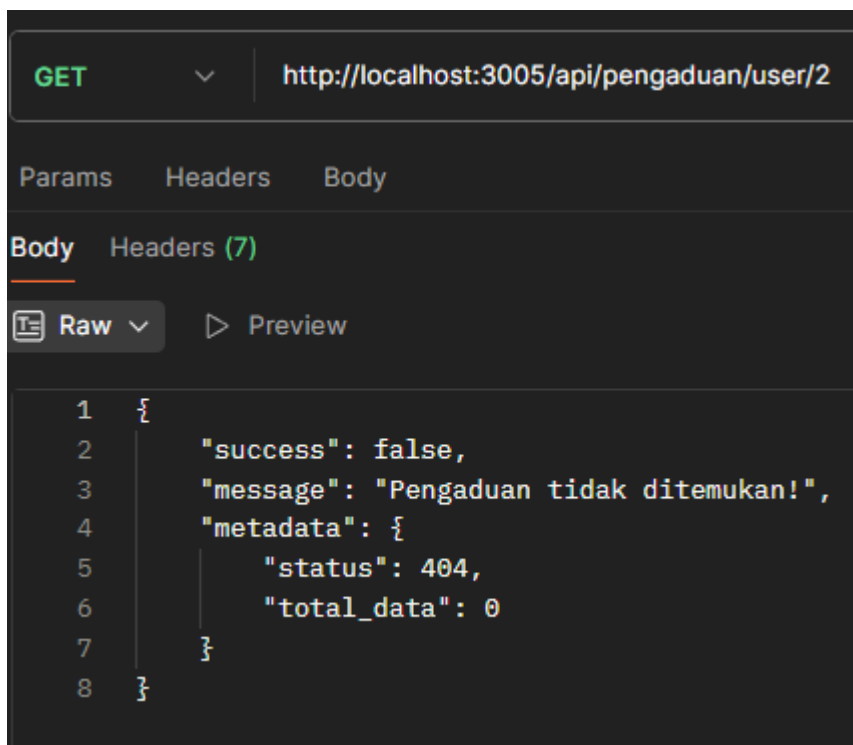
- GET by User ID



The screenshot shows a REST client interface with a GET request to `http://localhost:3005/api/pengaduan/user/1`. The response body is displayed in raw format, showing a successful status with a message and a list of complaint data.

```
1 {
2   "success": true,
3   "message": "Pengaduan berhasil ditemukan.",
4   "metadata": {
5     "status": 200,
6     "total_data": 1
7   },
8   "data": [
9     {
10      "id": 1,
11      "user_id": "1",
12      "judul": "Jalan rusak di depan rumah",
13      "deskripsi": "Jalan di RT 03 sudah rusak parah dan berbahaya untuk kendaraan",
14      "status": "MENUNGGU",
15      "created_at": "2026-05-06T03:38:15.579Z",
16      "updated_at": "2026-05-06T03:38:15.579Z"
17    }
18  ]
19 }
```

Ini adalah respon dari GET pengaduan by User ID. Request ini digunakan untuk mengkategorikan pengaduan berdasarkan Id dari user. Untuk menggunakan request ini, kita perlu menambahkan [pengaduan/user/[id]] id dari user di url.

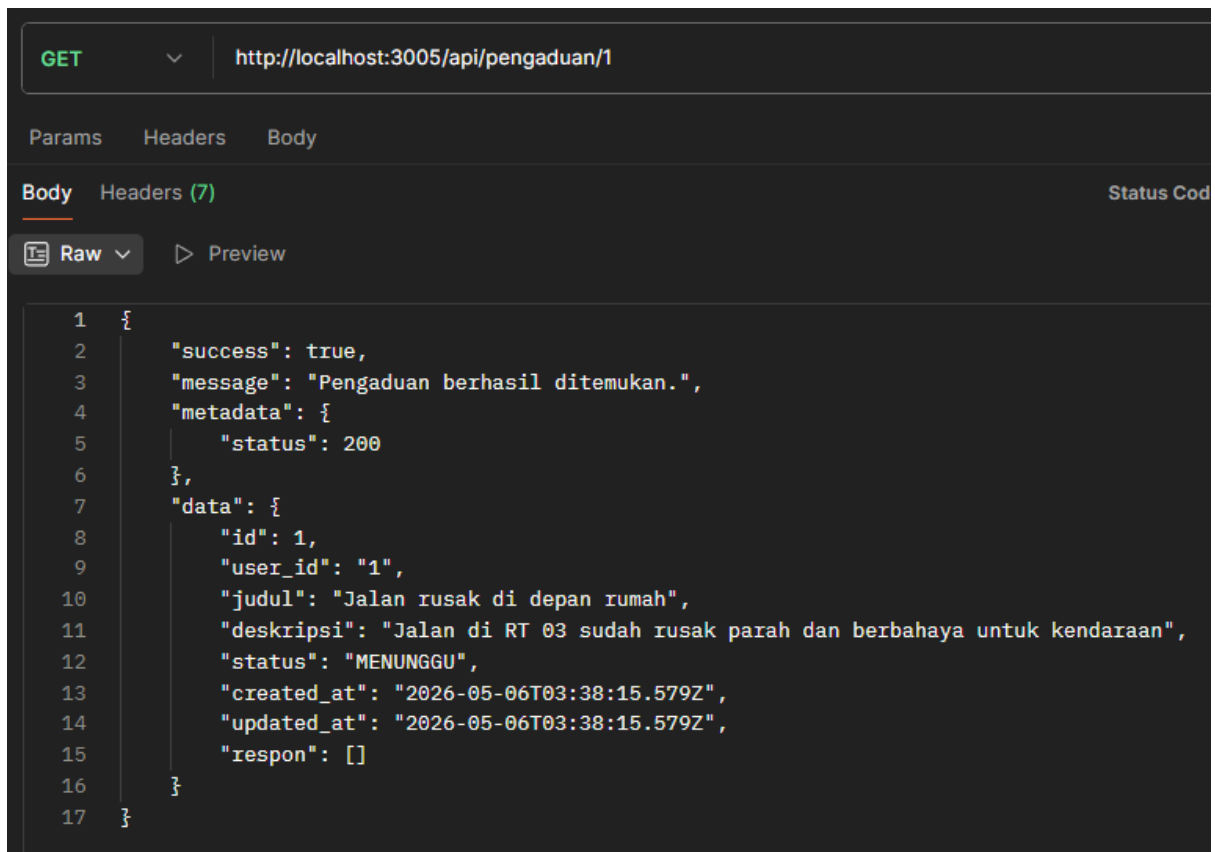


The screenshot shows a REST client interface with a GET request to `http://localhost:3005/api/pengaduan/user/2`. The response body is displayed in raw format, showing a failed status with a message indicating the complaint was not found.

```
1 {
2   "success": false,
3   "message": "Pengaduan tidak ditemukan!",
4   "metadata": {
5     "status": 404,
6     "total_data": 0
7   }
8 }
```

Jika id dari user tidak ada di database, maka pengaduan juga tidak akan ditemukan.

- GET by ID



The screenshot shows a REST client interface with a GET request to `http://localhost:3005/api/pengaduan/1`. The response body is displayed in raw JSON format, showing a successful status with a message and detailed complaint data.

```
1 {
2   "success": true,
3   "message": "Pengaduan berhasil ditemukan.",
4   "metadata": {
5     "status": 200
6   },
7   "data": {
8     "id": 1,
9     "user_id": "1",
10    "judul": "Jalan rusak di depan rumah",
11    "deskripsi": "Jalan di RT 03 sudah rusak parah dan berbahaya untuk kendaraan",
12    "status": "MENUNGGU",
13    "created_at": "2026-05-06T03:38:15.579Z",
14    "updated_at": "2026-05-06T03:38:15.579Z",
15    "respon": []
16  }
17 }
```

Ini adalah respon dari pengaduan GET by ID. Di request ini hanya akan memanggil data dengan id yang sudah ada, jika id dari pengaduan tidak ada di database, maka pengaduan tidak akan ditemukan. Untuk menggunakan request ini kita perlu menambahkan id dari pengaduan di url.

- PATCH

PATCH ▼ `http://localhost:3005/api/pengaduan/1/status`

Params Headers **Body** ●

☐ none ☐ form-data ☒ x-www-form-urlencoded ☐ raw ☐ binary

	Key	Value
☒	status	DIPROSES
☐	status	SELESAI
	Key	Value

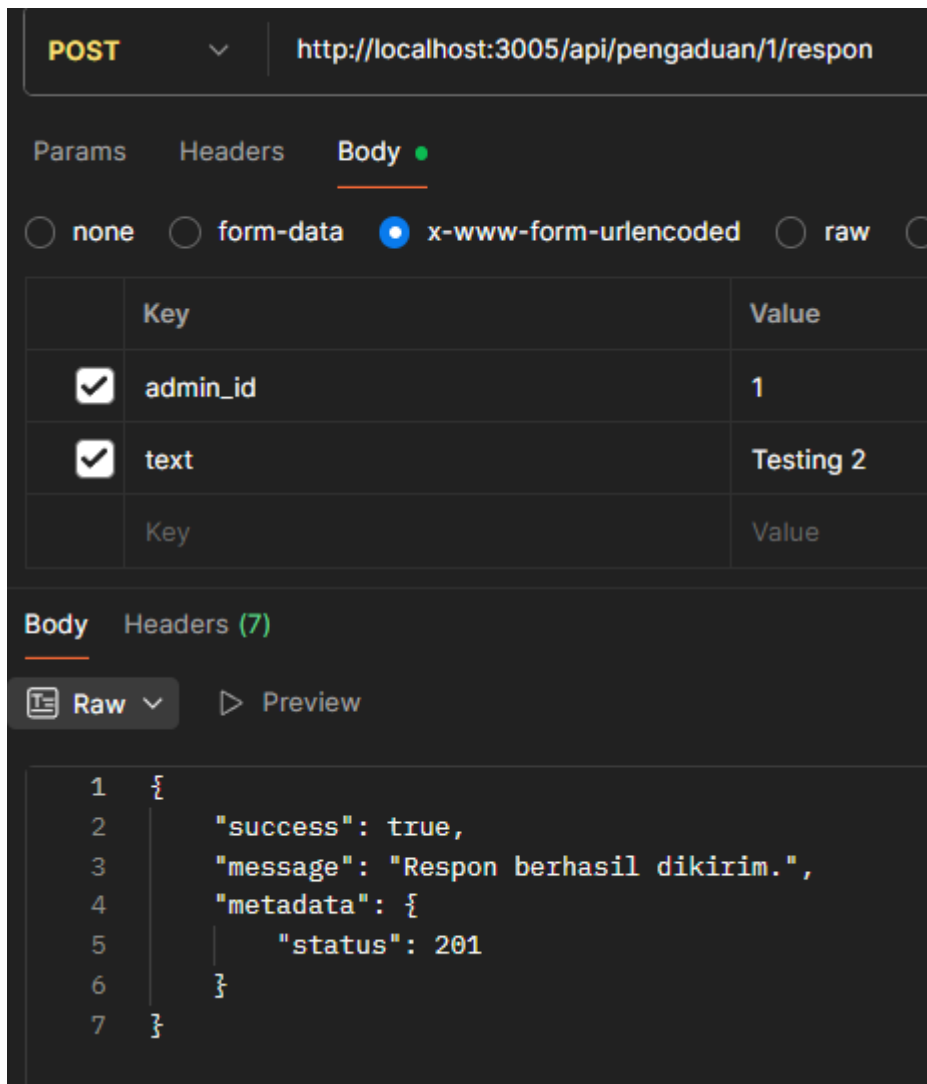
Body Headers (7)

☒ Raw ▼ ▶ Preview

```
1 {  
2   "success": true,  
3   "message": "Status pengaduan berhasil diupdate.",  
4   "metadata": {  
5     "status": 200  
6   }  
7 }
```

Ini adalah respon dari PATCH status pengaduan. Untuk PATCH pengaduan hanya bisa di akses oleh admin, dikarenakan hanya untuk mengubah status dari pengaduan apakah sudah DIPORSES atau masih MENUNGGU ataupun juga sudah SELESAI. Untuk menggunakan request ini kita perlu memasukkan [{id}/status] pada url. Dan untuk body hanya perlu memasukkan status DIPROSES/SELESAI.

- POST Respon



The screenshot displays a REST client interface for a POST request. The URL is `http://localhost:3005/api/pengaduan/1/respon`. The request body is configured as `x-www-form-urlencoded` with two parameters: `admin_id` (value: `1`) and `text` (value: `Testing 2`). The response is shown in raw JSON format.

Request Parameters:

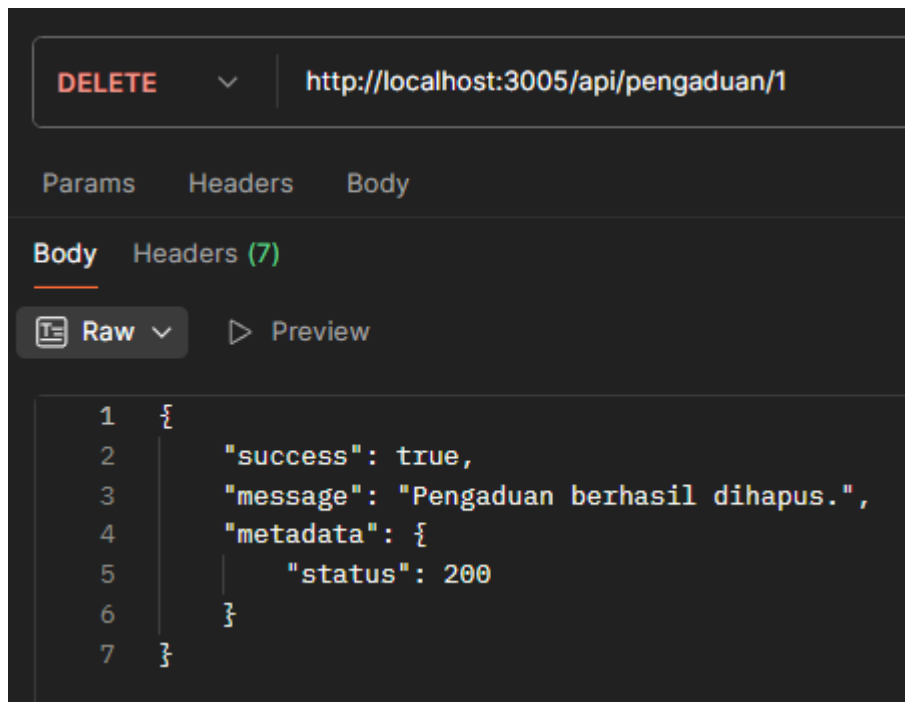
Key	Value
☒ <code>admin_id</code>	<code>1</code>
☒ <code>text</code>	<code>Testing 2</code>

Response Body (Raw):

```
1 {
2   "success": true,
3   "message": "Respon berhasil dikirim.",
4   "metadata": {
5     "status": 201
6   }
7 }
```

Ini adalah respon dari POST Respon. Request ini digunakan untuk membuat respon yang dikirimkan kepada user yang sudah membuat pengaduan. Apakah ada kesalahan pada proses pembuatan pengaduan, sehingga user bisa tau salah nya dimana.

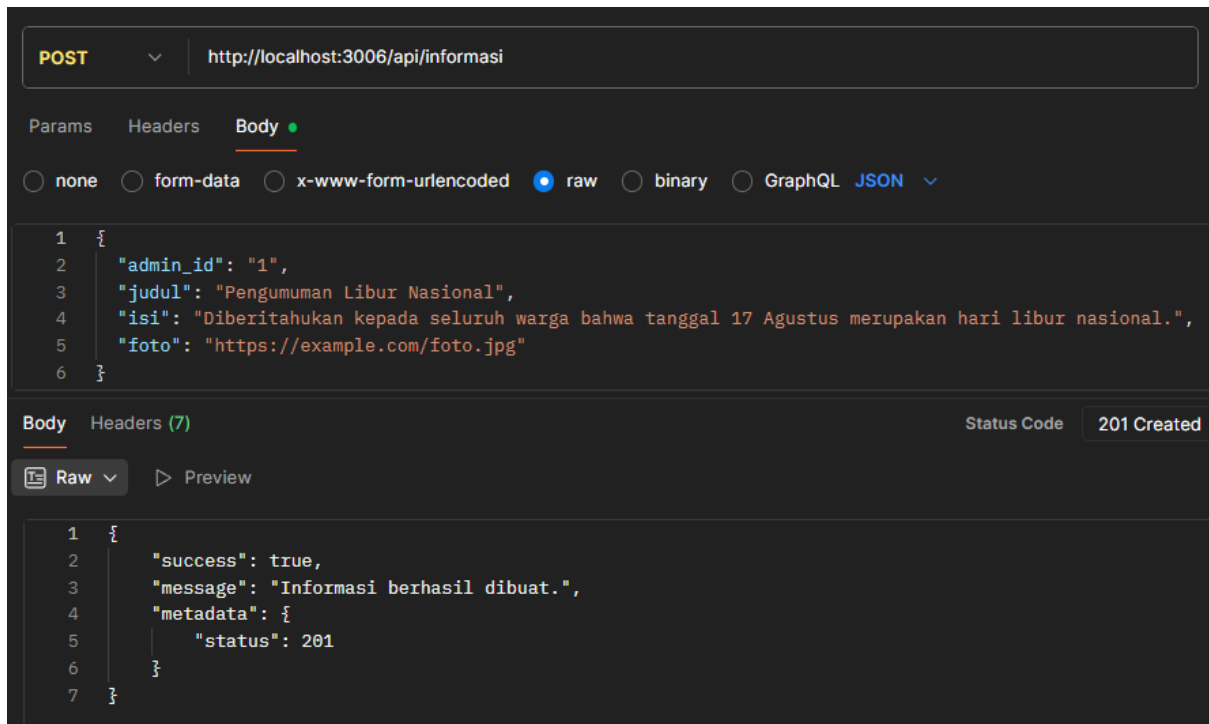
- DELETE



Ini adalah respon dari DELETE pengaduan. Disini kita hanya perlu memasukkan id dari pengaduan yang ingin kita hapus.

API/informasi

- POST



Ini adalah respon dari POST Informasi. Request ini digunakan untuk membuat sebuah informasi yang berisi

- judul
- isi
- url_foto

- GET ALL

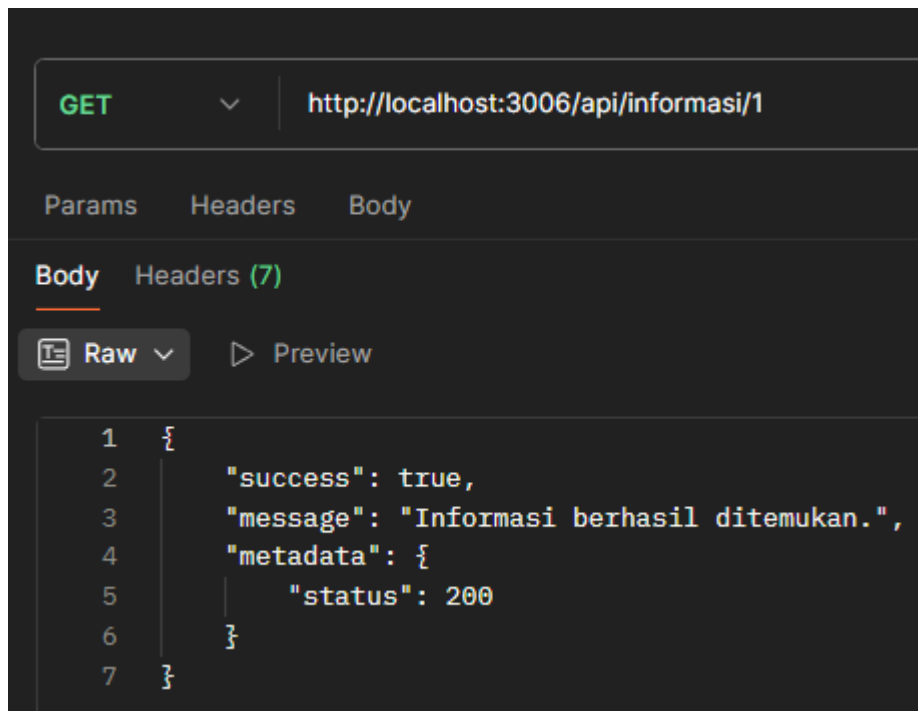
The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:3006/api/informasi
- Status Code:** 200 OK
- Response Body (Raw):**

```
1 {
2   "success": true,
3   "message": "Informasi berhasil ditemukan.",
4   "metadata": {
5     "status": 200,
6     "total_data": 1
7   },
8   "data": [
9     {
10      "id": 1,
11      "admin_id": "1",
12      "judul": "Pengumuman Libur Nasional",
13      "isi": "Diberitahukan kepada seluruh warga bahwa tanggal 17 Agustus merupakan hari libur nasional.",
14      "foto": "https://example.com/foto.jpg",
15      "created_at": "2026-05-06T07:49:09.360Z",
16      "updated_at": "2026-05-06T07:49:09.360Z"
17    }
18  ]
19 }
```

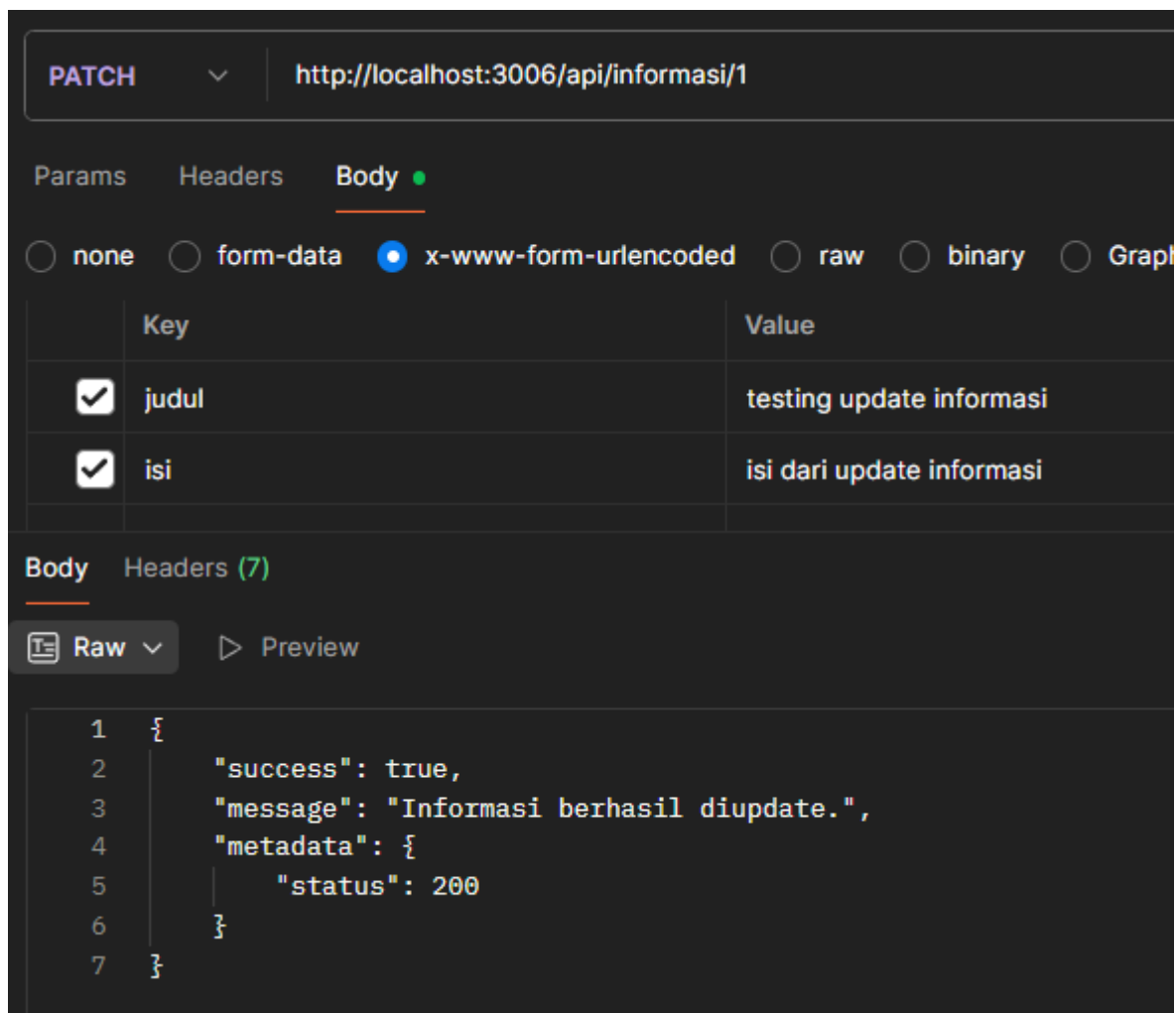
Ini adalah respon dari GET informasi. Disini tidak seperti post yang membutuhkan body atau data untuk dikirimkan, kita hanya perlu memasukkan url dari request ini.

- GET by ID



Ini adalah respon dari informasi GET by ID. Di request ini hanya akan memanggil data dengan id yang sudah ada, jika id dari informasi tidak ada di database, maka informasi tidak akan ditemukan. Untuk menggunakan request ini kita perlu menambahkan id dari informasi di url.

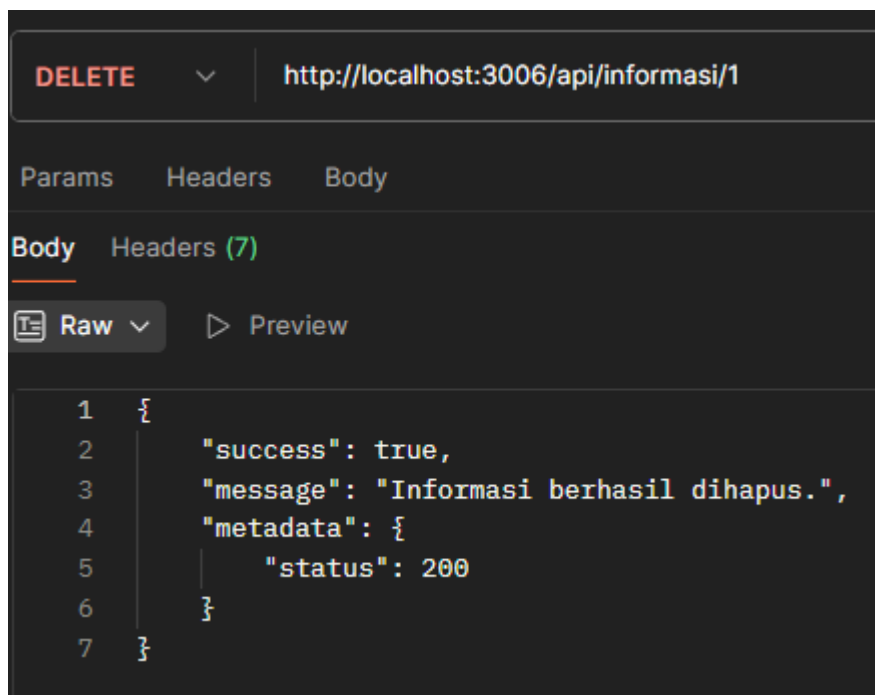
- PATCH



Ini adalah respon dari PATCH informasi. Disini kita hanya perlu memasukkan id dari informasi yang ingin di ubah datanya, lalu body yang untuk mengganti data yang ingin kita ubah seperti

- judul
- isi

- DELETE



Ini adalah respon dari DELETE informasi. Disini kita hanya perlu memasukkan id dari informasi yang ingin kita hapus.

- POST foto

The screenshot shows a REST client interface with the following details:

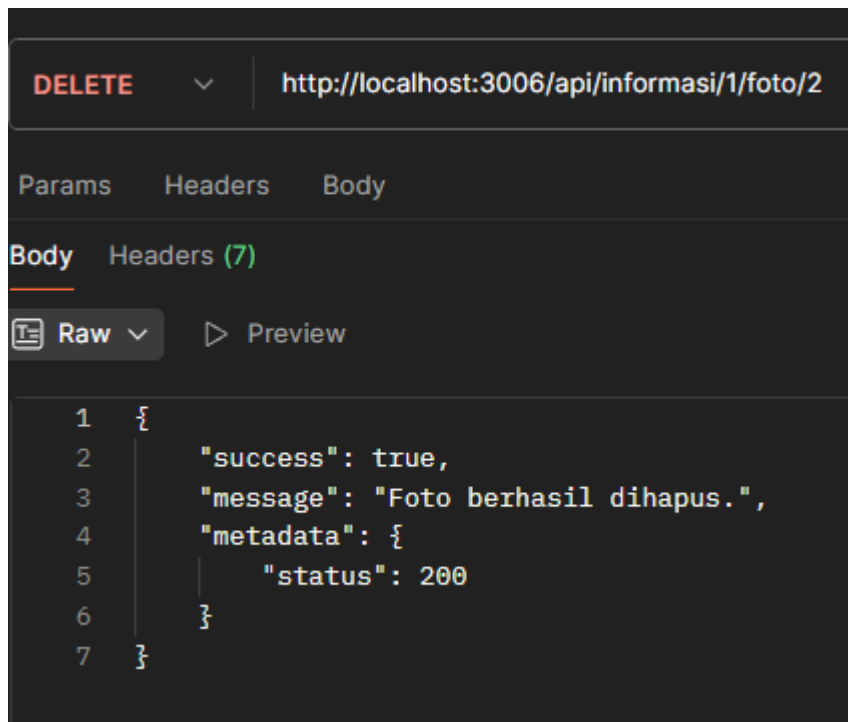
- Method:** POST
- URL:** http://localhost:3006/api/informasi/1/foto
- Body Type:** form-data (selected)
- Form Data:**

Key	Value
foto	IMG_20251231_090244.jpg
- Response Body (Raw):**

```
1  {
2    "success": true,
3    "message": "Foto berhasil ditambahkan.",
4    "metadata": {
5      "status": 201
6    },
7    "data": {
8      "id": 2,
9      "informasi_id": 1,
10     "filename": "foto-1778223169561-388946580.jpg",
11     "created_at": "2026-05-08T06:52:49.626Z"
12   }
13 }
```

Ini adalah respon dari POST foto untuk informasi. Disini kita hanya perlu memasukkan [{id}/foto] pada url, lalu untuk body nya kita hanya perlu memilih foto yang ingin di masukkan ke database milik informasi.

- DELETE foto

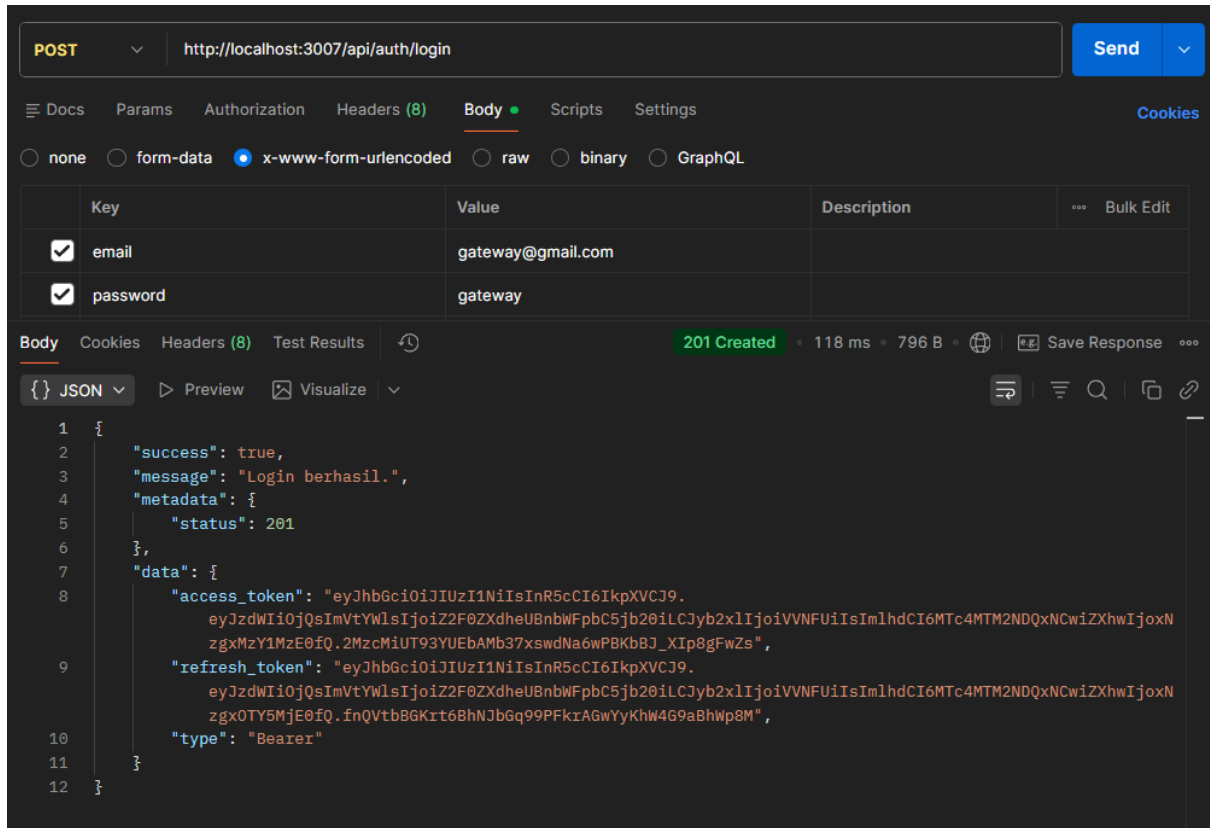


Ini adalah respon dari DELETE foto. Untuk menghapus foto dari sebuah informasi, kita perlu :

- /id_informasi
- /foto
- /id_foto

API/Gateway

- AUTH/login



The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:3007/api/auth/login
- Body Type:** x-www-form-urlencoded
- Body Data:**

Key	Value
email	gateway@gmail.com
password	gateway
- Status:** 201 Created
- Response Time:** 118 ms
- Response Size:** 796 B
- Response Body (JSON):**

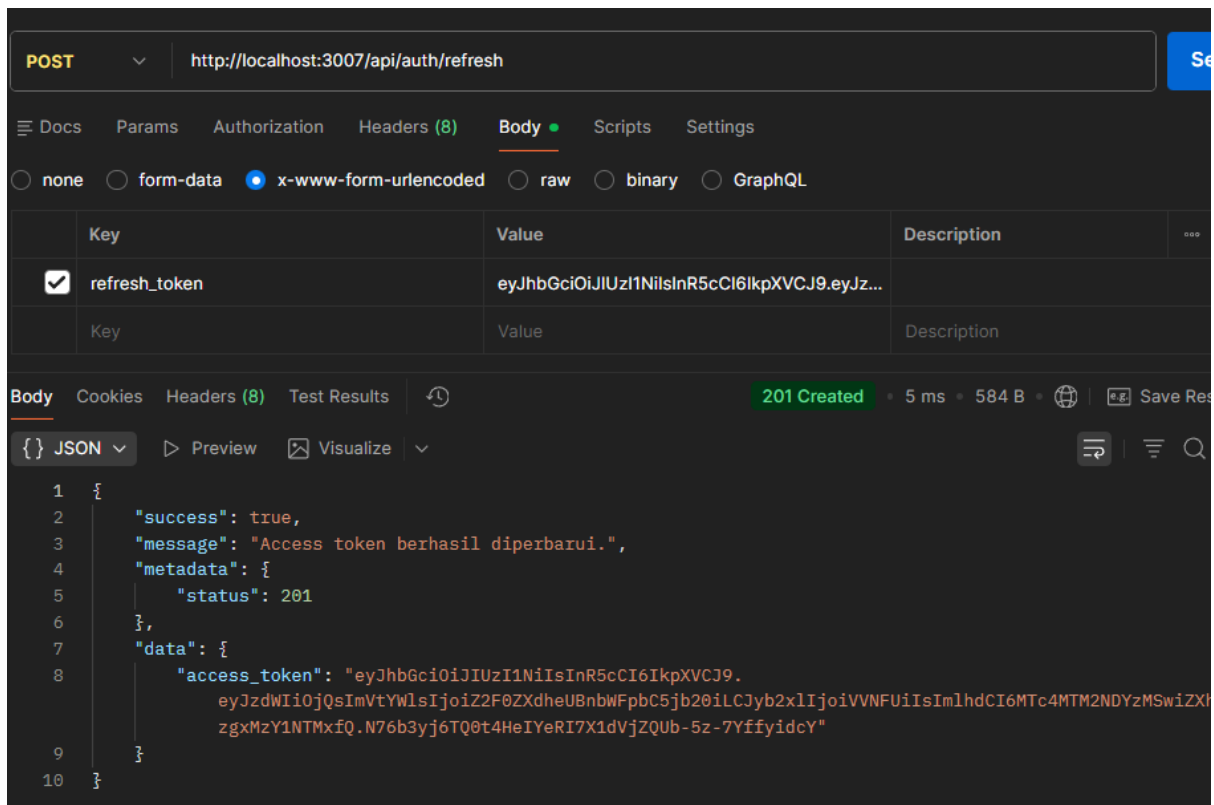
```
{  "success": true,  "message": "Login berhasil.",  "metadata": {    "status": 201  },  "data": {    "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJsImVtYWlsIjoiz2F0ZXdheUBnbWZpb20iLCJyb2xlIjoivVNFUiIsIm1hdCI6MTc4MTM2NDQxNCwiZXhwIjoxNzgxMzY1MzE0fQ.2MzcMiUT93YUEbAMB37xswdNa6wPBKbBJ_XIp8gFwZs",    "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJsImVtYWlsIjoiz2F0ZXdheUBnbWZpb20iLCJyb2xlIjoivVNFUiIsIm1hdCI6MTc4MTM2NDQxNCwiZXhwIjoxNzgxOTY5MjE0fQ.fnQVtb8GKrt68hNjbGq99PFkrAGWYyKhW4G9aBhWp8M",    "type": "Bearer"  }  }
```

Ini adalah respon dari system login pada gateway. Untuk bisa login kita memerlukan

- Email
- Password

Setelah kita berhasil login respon dari request login akan mengirimkan berupa pesan, lalu ada access_token, refresh_token, type_bearer. Access Token digunakan untuk membuka akses pada service lainnya, yang menandakann ada usern yang sedang login.

- AUTH/refresh



The screenshot shows a REST client interface with the following details:

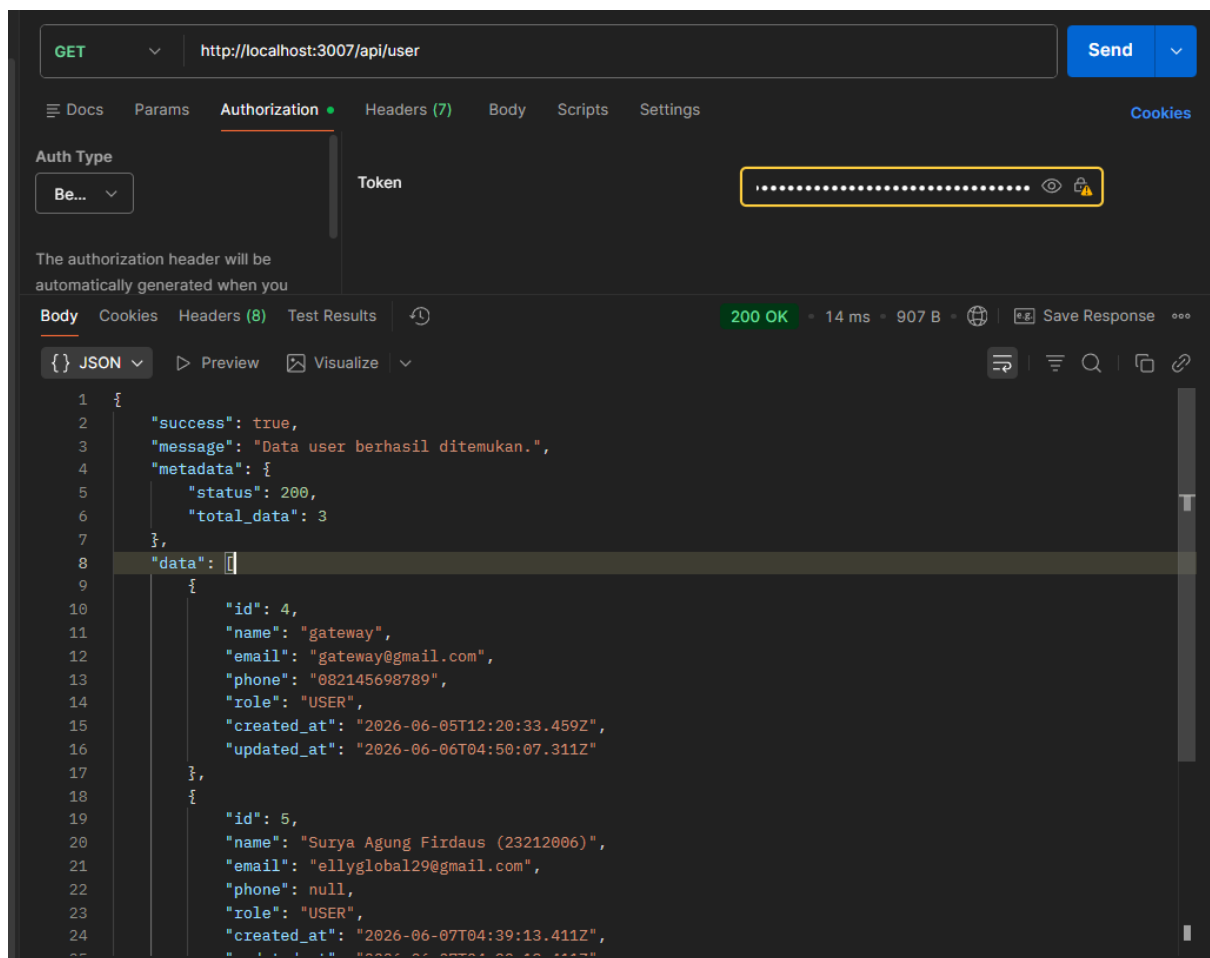
- Method:** POST
- URL:** http://localhost:3007/api/auth/refresh
- Body Type:** x-www-form-urlencoded
- Body Content:**

Key	Value	Description
refresh_token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJz...	
- Response:** 201 Created, 5 ms, 584 B
- Response Body (JSON):**

```
1 {
2   "success": true,
3   "message": "Access token berhasil diperbarui.",
4   "metadata": {
5     "status": 201
6   },
7   "data": {
8     "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzZdWIiOiJQsImVtYWlsIjoizZ2F0ZXdhZU8nbWFpbC5jb20iLCJyb2x1IjoivVNFUiIsIm1hdCI6MTc4MTM2NDYzMSwiZXNzIjozY1NTMxZQ.N76b3yj6TQ0t4HeIYeRI7X1dVjZQub-5z-7YffydY"
9   }
10 }
```

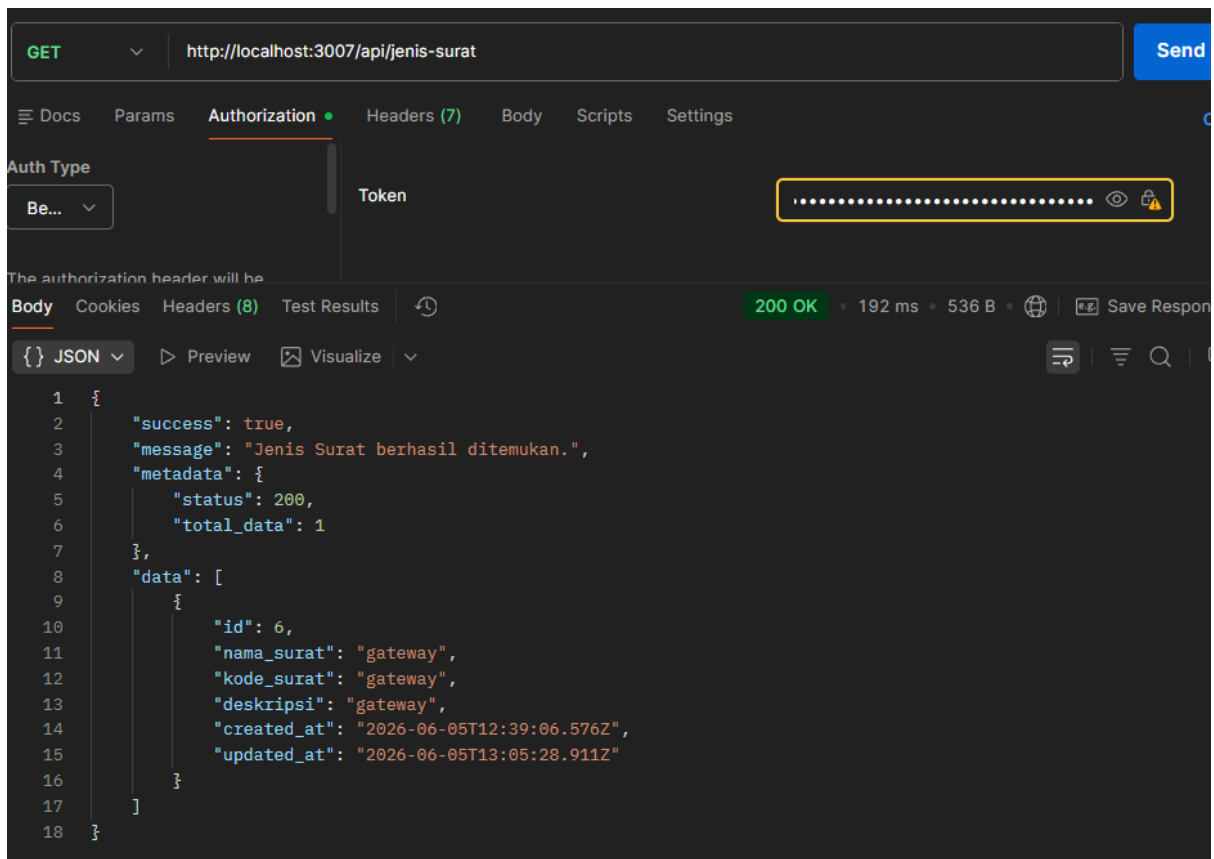
Ini adalah respon dari refresh. Request ini digunakan untuk memperbarui access_token yang sudah expired, kita hanya perlu memasukkan refresh_token yang dikirimkan di respon login sebelumnya. Setelah memasukkan refresh_token, system akan mengirimkan access_token baru yang siap digunakan.

- GATEWAY/user



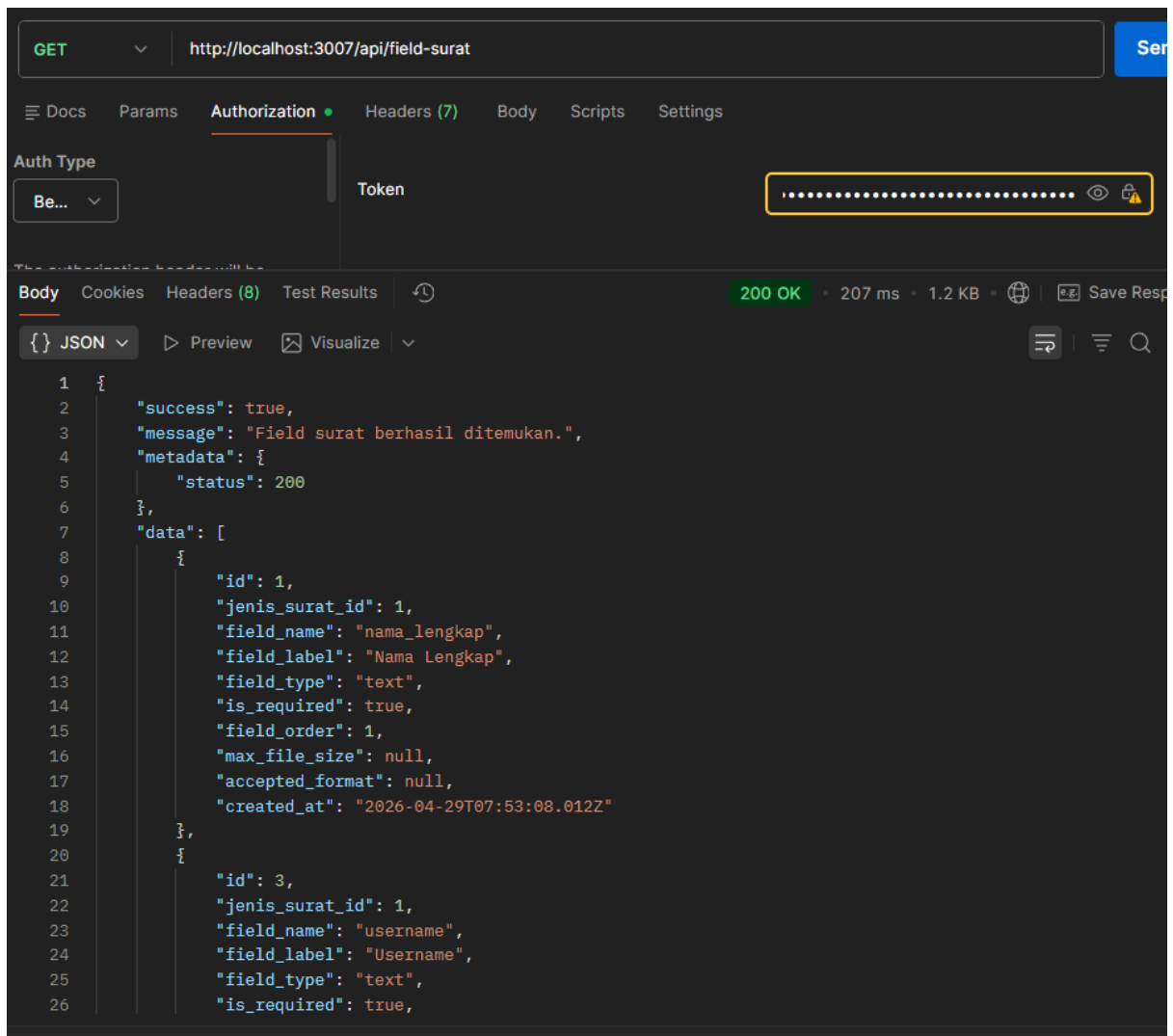
Sistem user service yang ada di gateway bekerja dengan cara yang sama seperti sebelumnya. Tetapi perbedaan disini kita memerlukan token Bearer untuk mengakses seluruh request milih user-service ini.

- GATEWAY/jenis-surat



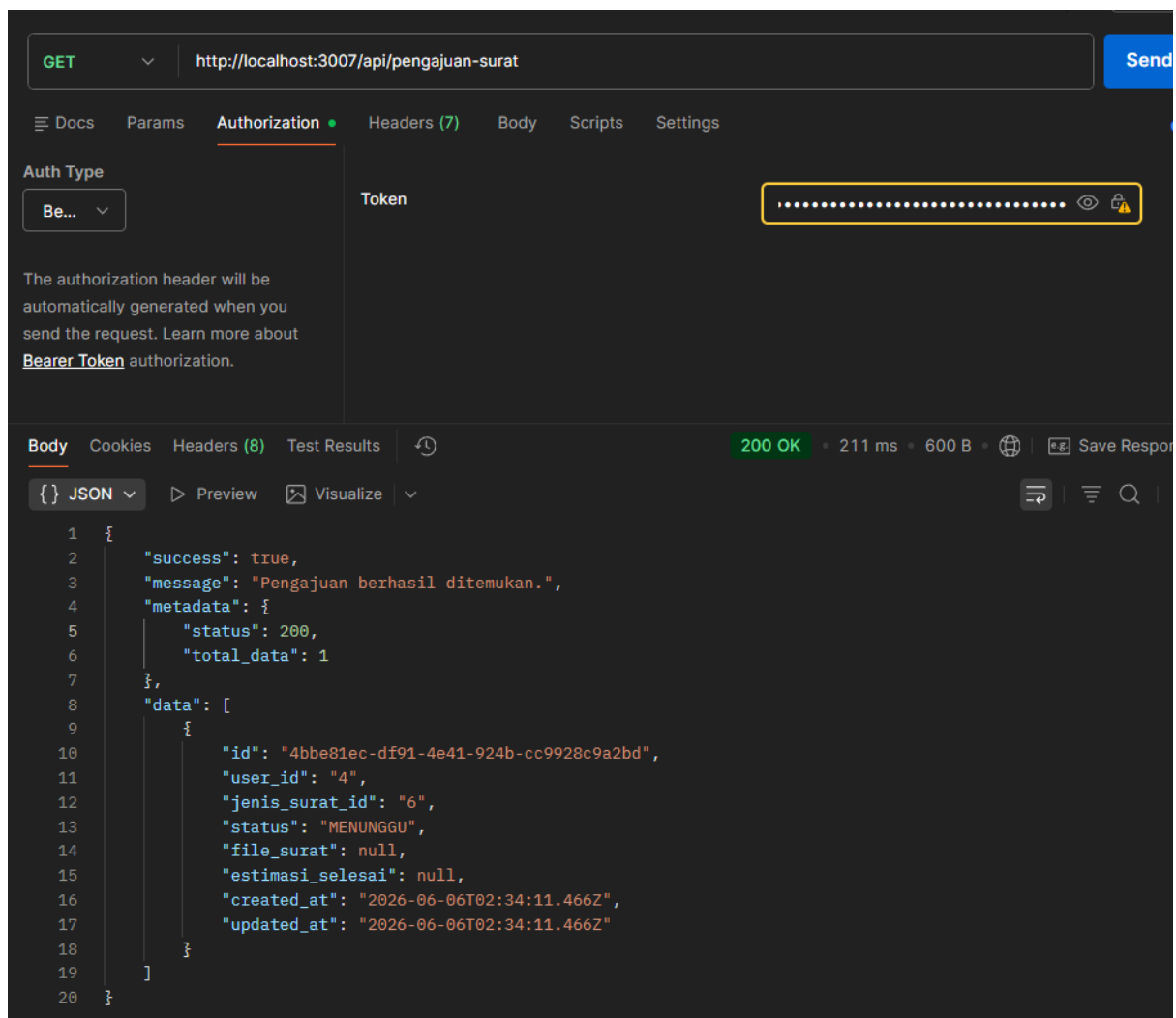
Sama seperti gateway/user, untuk mengakses jenis-surat kita memerlukan token bearer.

- GATEWAY/field-surat



Sama seperti sebelumnya, untuk menggunakan request field-surat kita memerlukan token bearer.

- GATEWAT/pengajuan-surat



Untuk menggunakan request service pengajuan-surat, kita juga membutuhkan token bearer.

- GATEWAY/pengaduan

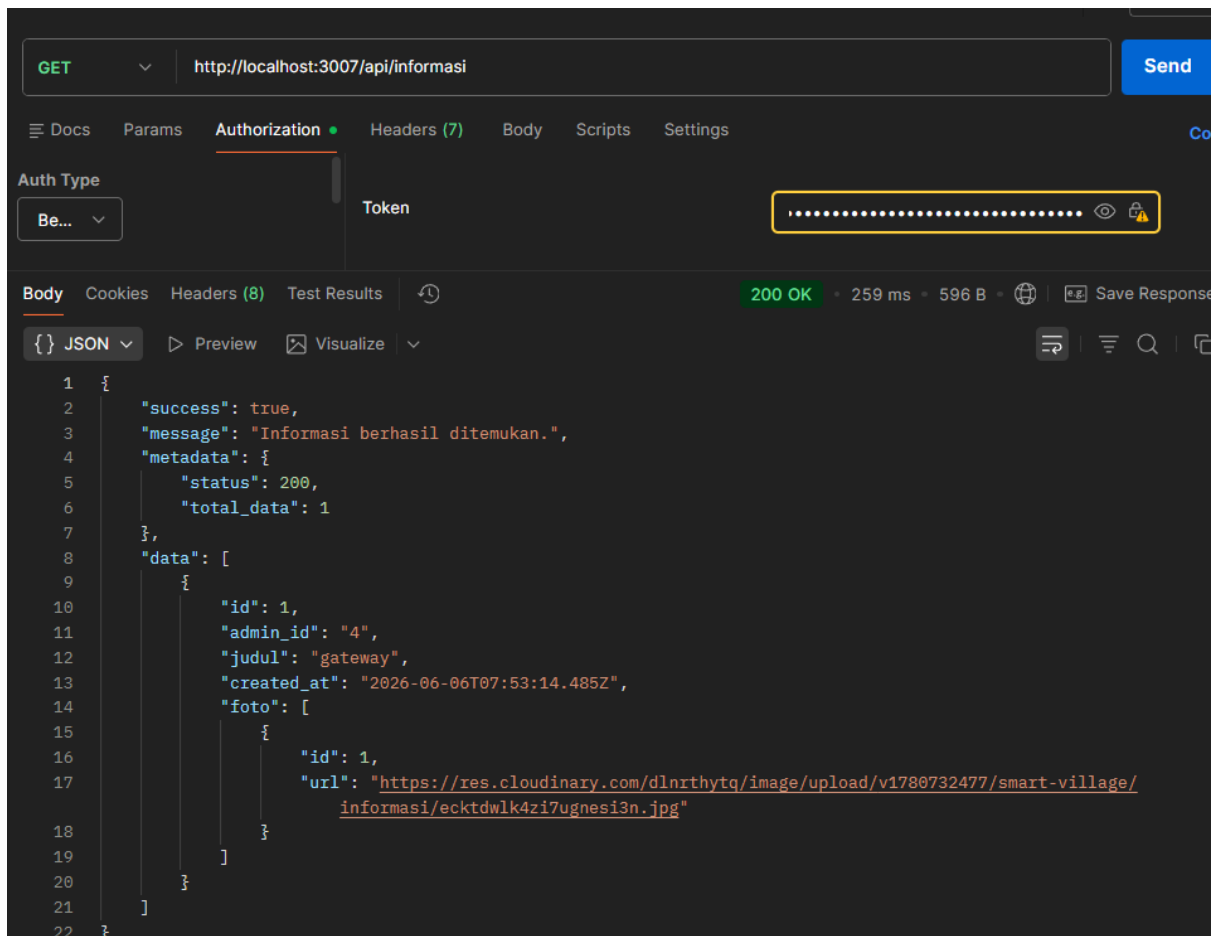
The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:3007/api/pengaduan
- Auth Type:** Bearer Token (Token field is masked with dots)
- Status:** 200 OK
- Response Time:** 244 ms
- Response Size:** 678 B
- Body (JSON):**

```
1 {
2   "success": true,
3   "message": "Pengaduan berhasil ditemukan.",
4   "metadata": {
5     "status": 200,
6     "total_data": 2
7   },
8   "data": [
9     {
10      "id": 4,
11      "user_id": "4",
12      "judul": "Gateway",
13      "status": "SELESAI",
14      "created_at": "2026-06-06T04:57:08.977Z",
15      "updated_at": "2026-06-06T05:06:47.810Z"
16    },
17     {
18      "id": 2,
19      "user_id": "10",
20      "judul": "Jalan rusak di depan rumah",
21      "status": "MENUNGGU",
22      "created_at": "2026-06-06T04:45:48.590Z",
23      "updated_at": "2026-06-06T04:45:48.590Z"
24    }
25  ]
26 }
```

Sama seperti sebelumnya, untuk menggunakan request pengaduann kita memerlukan token bearer.

- GATEWAY/informasi



Sama seperti sebelumnya, untuk menggunakan request informasi kita memerlukan token bearer.